

Conditional Drifting Models for Learned Channel Simulation

Rick Fritschek and Rafael F. Schaefer
Chair of Information Theory and Machine Learning
Technische Universität Dresden, Germany

Abstract—Accurate stochastic channel simulation is central to data-driven communication system design, especially when channel measurements are costly to acquire repeatedly or when differentiable learned surrogates are needed inside end-to-end optimization loops. Recent work has shown that conditional diffusion models are strong channel generators, but their inference cost scales with the number of reverse denoising steps. This paper presents an experimental study of *conditional drifting models* for data-driven channel modeling and evaluates them as a low-latency alternative to conditional DDPM, DDIM, and WGAN baselines. We study both residual and direct-output drifting on AWGN, Rayleigh fading, SSPA nonlinearity, and a simplified optical-fiber proxy channel. In addition, we consider an enhanced direct-output variant with a conditioning-aware kernel. The results show that diffusion achieves the strongest sliced Wasserstein distance (SWD) overall, while direct-output drifting is the strongest one-shot generator under direct output-space evaluation. A symbolic AWGN case study further shows that direct drifting can benefit from conditioning-aware kernel design on downstream SER-based metrics. Timing results confirm a persistent microsecond-versus-millisecond inference gap between one-shot drifting and iterative diffusion sampling. These results position drifting as a promising fast alternative to iterative diffusion-based channel generators.

I. INTRODUCTION

Accurate channel models are a basic requirement for modern communication system design. They are particularly important in neural and end-to-end learned systems, where the channel appears repeatedly inside training loops, decoder adaptation procedures, and differentiable autoencoder pipelines. When a clean analytical channel model is unavailable, too restrictive, or poorly matched to measured data, generative channel surrogates become attractive because they can learn the full conditional distribution directly from samples.

This problem is not only about distributional accuracy. In many learning pipelines, the channel model is evaluated millions of times. As a result, sample quality, conditioning accuracy, computational cost, and differentiability all matter simultaneously. The question is therefore not merely whether a generative model can match the channel distribution, but whether it can do so at a cost compatible with large-scale training and deployment workflows.

Recent work on diffusion-based channel modeling has shown that conditional diffusion models can generate high-quality channel samples across standard benchmarks such as additive white Gaussian noise (AWGN), Rayleigh fading, and nonlinear amplifier channels [1]. Follow-up work has further

studied robustness and speed-quality tradeoffs for diffusion-based channel generation [2], extended diffusion-based channel synthesis to high-dimensional user-specific channels [3], and proposed diffusion-driven digital-twin-style generation of statistical channel state information from sensing and location information [4]. More broadly, diffusion methods have also begun to influence adjacent wireless tasks such as high-dimensional channel estimation [5], low-complexity multiple-input multiple-output (MIMO) channel estimation [6], [7], joint channel estimation and data detection [8], and fast time-varying MIMO orthogonal frequency-division multiplexing (MIMO-OFDM) estimation [9]. This line of work is compelling because diffusion models are flexible, robust, and often more stable than earlier adversarial approaches. However, this improvement comes with a practical system cost: inference requires an iterative reverse denoising procedure, and the latency scales with the number of diffusion steps. This tradeoff is manageable when the channel generator is called occasionally, but it becomes more problematic when it is embedded in inner loops of iterative training or repeated Monte Carlo evaluations. This latency bottleneck has motivated several acceleration strategies for diffusion, including skipped or reduced-step sampling as in denoising diffusion implicit modeling (DDIM), as well as one-step or few-step alternatives such as consistency models and distillation-based approaches that compress multi-step samplers into substantially shorter generation paths [10]–[12]. This motivates the study of one-shot conditional generators for learned channel simulation. Among recently proposed generative paradigms, drifting models [13] are particularly interesting because they aim to learn the transport from a simple latent source to the target data distribution directly in the network parameters, rather than unfold that transport as a multi-step sampling trajectory at inference time. Intuitively, diffusion spends computation at test time by progressively transforming noise into data through a reverse-time chain. Drifting, in contrast, spends computation during training: Generated samples are repeatedly nudged toward the data manifold by a drift field, and the neural generator is updated so that future samples move closer to the desired distribution in a single forward pass. Once trained, sampling is therefore naturally one-shot.

That distinction makes the concept of drifting conceptually appealing for channel modeling. In a communication setting, the generator must be conditional on the transmitted symbol x , and it must capture not just a marginal sample

cloud, but the conditional law $p(y|x)$. Because drifting is based on attraction and repulsion between generated and true samples rather than an adversarial discriminator game, it is also an interesting candidate for studying whether one-shot conditional generators can avoid some of the mode-collapse behavior often associated with adversarial training. Because drifting was introduced only very recently, no widely adopted communication-channel baseline has yet emerged. This motivates a benchmark-oriented study that reports not only comparative results, but also the concrete conditional adaptation needed to integrate drifting into an existing channel-learning pipeline.

The emphasis of this study is on generator-level distributional accuracy and computational tradeoffs rather than on downstream symbol-error-rate curves. Accordingly, we focus on low-dimensional conditional channel benchmarks that isolate generator behavior before moving to more realistic settings with memory, MIMO structure, or higher-dimensional conditioning.

The main contributions are summarized as follows.

- We formulate conditional drifting generators for channel simulation in both residual form, using $e = y - x$, and direct-output form, modeling $p(y|x)$ directly.
- We provide a practical drifting objective based on attraction and repulsion in the chosen target space, together with an enhanced direct-output variant that uses a conditioning-aware kernel on joint condition-target pairs.
- We compare the resulting drifting variants against conditional denoising diffusion probabilistic modeling (DDPM), conditional DDIM, and conditional Wasserstein generative adversarial network (WGAN) models on AWGN, Rayleigh fading, solid state power amplifier (SSPA), and optical-fiber (OptFib) proxy channels.
- We quantify the accuracy–latency tradeoff and show that one-shot drifting provides large inference-time savings while remaining competitive in distributional accuracy. A symbolic AWGN case study further illustrates that conditioning-aware kernel design can improve downstream direct-output drifting performance.

Code will be released upon publication¹.

II. PROBLEM FORMULATION AND DRIFTING BACKGROUND

A. What Drifting Changes Conceptually

Because drifting is very recent and remains less familiar than diffusion, we briefly summarize its generative viewpoint. Following [13], let f_θ map a latent variable $z \sim p_z$ to a sample $u = f_\theta(z)$, and let

$$q_\theta = (f_\theta)_\# p_z \quad (1)$$

denote the pushforward distribution induced by the generator. The training objective is no longer expressed as a likelihood surrogate or as a denoising score-matching loss. Instead, the paper introduces a *drifting field* that tells us how samples

drawn from the current model distribution should move so that the model distribution approaches the target distribution p .

At a high level, the desired property is an equilibrium condition: If the model already matches the target distribution, then the drift should vanish. Using $V_{p,q}(u)$ to denote the field evaluated at a sample u under target distribution p and model distribution q , the fixed-point intuition can be summarized as

$$q = p \implies V_{p,q}(u) = 0, \quad (2)$$

which is the sense in which matched distributions form an equilibrium of the training dynamics [13].

This makes the training logic very different from diffusion. In diffusion, one learns a denoiser and then performs iterative refinement at inference time. In drifting, one instead constructs improved targets during training by moving current samples along the field $V_{p,q}$, and then regresses the generator onto those improved targets. As a result, the iterative correction happens during optimization rather than at test time.

The one-step view can be written as follows. Starting from a generated sample $u = f_\theta(z)$, define a drifted target

$$\tilde{u} = \text{stopgrad}(u + \eta V_{p,q_\theta}(u)), \quad (3)$$

and optimize the generator by

$$\mathcal{L}_{\text{drift}} = \mathbb{E}_{z \sim p_z} [\|f_\theta(z) - \tilde{u}\|_2^2]. \quad (4)$$

The stop-gradient is essential here: It makes the drifted sample behave like a fixed target for the current optimization step. Repeated training steps therefore evolve the pushforward q_θ toward the target distribution. This is also the cleanest way to see why drifting is naturally one-shot at inference: Once the generator has learned to emit samples near equilibrium, sampling only requires drawing z and evaluating $f_\theta(z)$, with no reverse-time chain.

Fig. 1 gives an actual toy particle simulation in the spirit of the particle-based demonstrations used in [13]. For two fixed channel inputs, we initialize mismatched conditional residual particle clouds and then evolve them under the same attraction–repulsion drift rule used in our channel benchmark. The figure, therefore, illustrates real drift dynamics rather than a hand-drawn schematic, while still serving only as a toy analogue of the full learned generator. To show that the same idea can train an explicit parameterized generator, Fig. 2 uses a tiny conditional MLP trained with the drifting objective on the same toy target.

B. Conditional Channel Simulation

We now specialize the generic drifting view to learned conditional channel simulation. Let $x \in \mathbb{R}^n$ denote the transmitted channel input and $y \in \mathbb{R}^n$ the channel output. The objective is to learn a conditional generator that reproduces the channel law given x . In this setting, conditioning enters through the transmitted symbol, while stochasticity is carried by a latent variable $z \sim \mathcal{N}(0, I)$.

Two target parameterizations are useful in the experiments. The first is the direct channel-output formulation, in which

¹Code repository: <https://github.com/Fritschek>.

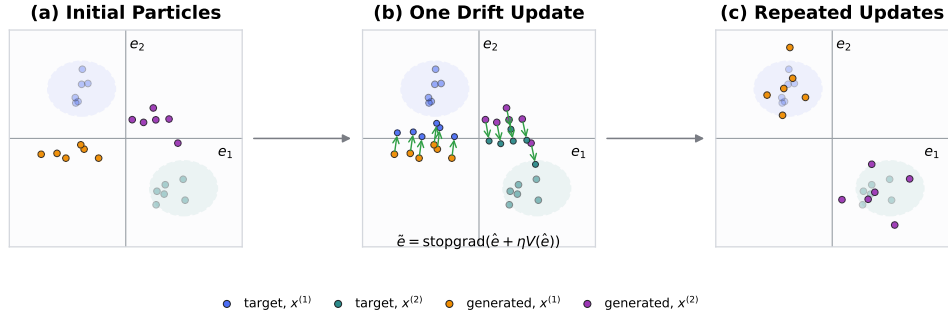


Fig. 1. Toy conditional particle simulation in residual space. Two different channel inputs induce two different target residual distributions. Starting from mismatched generated particle clouds, the middle panel applies one attraction–repulsion drift update of the form $\tilde{e} = \text{stopgrad}(\hat{e} + \eta V(\hat{e}))$, and the last panel shows the particle clouds after repeated updates. This figure is an actual toy simulation of the drift field, not a hand-drawn schematic.

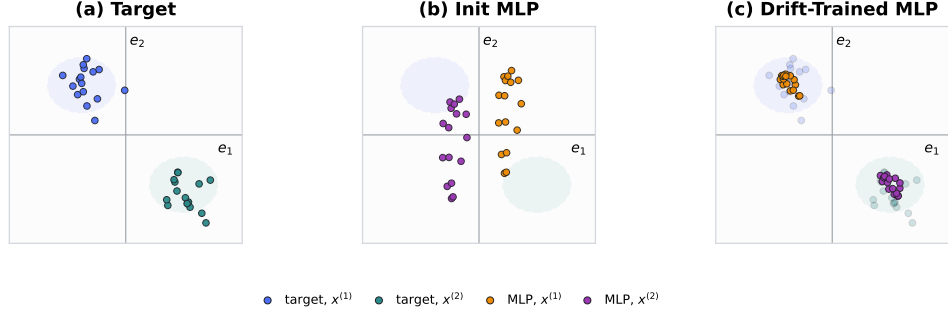


Fig. 2. Toy conditional MLP trained with the drifting objective. The left panel shows target conditional residual samples. The middle panel shows samples from a randomly initialized tiny conditional MLP $g_\theta(x, z)$. The right panel shows the same MLP after training with the drift objective, where the generated samples move close to the correct condition-dependent residual clouds.

the generator approximates $p(y|x)$ and predicts $\hat{y} = g_\theta(x, z)$. The second is the residual formulation, in which $e = y - x$ is modeled instead, so that the generator approximates $p(e|x)$ and the channel output is reconstructed as $\hat{y} = x + \hat{e}$. The direct formulation corresponds to learning the channel law itself, whereas the residual formulation injects the inductive bias that the channel output can be understood as the transmitted symbol plus a structured perturbation. As discussed later in the experiments, the direct formulation aligns naturally with direct-output diffusion and WGAN baselines, while the residual formulation is often easier to optimize but can fall short on nonlinear channels.

C. Conditional Drifting Model

We use a practical conditional adaptation inspired by [13]. To avoid confusion with the communication input symbol x , we use u above for the generic sample variable in the abstract formulation and reserve x below for the transmitted symbol. In our implementation, drifting can be applied either in the direct-output space or in the residual space. In the direct mode, the generator predicts $\hat{y} = g_\theta(x, z)$; in the residual mode, it predicts $\hat{e} = g_\theta(x, z)$ with reconstruction $\hat{y} = x + \hat{e}$, where $z \sim \mathcal{N}(0, I)$. In both cases, g_θ is implemented as a two-hidden-layer multilayer perceptron with hidden width 128,

latent dimension 16, and SiLU activations:

$$[x, z] \rightarrow \text{Linear} \rightarrow \text{SiLU} \rightarrow \text{Linear} \rightarrow \text{SiLU} \rightarrow \text{Linear}. \quad (5)$$

The training target is formed from an attraction–repulsion drift field in the chosen target space. This is a concrete channel-specific instantiation of the generic field $V_{p,q}$. Given generated samples $G = \{g_i\}$ and positive samples $P = \{p_j\}$, we compute

$$V(g_i) = \left(\sum_j w_{ij}^{(P)} p_j - g_i \right) - \lambda \left(\sum_k w_{ik}^{(G)} g_k - g_i \right), \quad (6)$$

where $w^{(P)}$ and $w^{(G)}$ are row-normalized RBF kernel weights. The first term attracts each generated sample toward a local barycenter of true samples in the chosen target space, and the second term repels it from nearby generated samples to reduce collapse. This yields a detached target

$$\tilde{g}_i = \text{stopgrad}(g_i + \eta V(g_i)), \quad (7)$$

and the generator is optimized by

$$\mathcal{L}_{\text{drift}} = \frac{1}{|G|} \sum_i \|g_i - \tilde{g}_i\|_2^2. \quad (8)$$

In other words, the generic update $u \mapsto u + \eta V_{p,q}(u)$ from the drifting formulation is realized here with a kernel-based

drift field estimated directly from the minibatches of true and generated samples in the selected target space. At a high level, this empirical drift field can be interpreted as a kernel particle interaction rule. The attraction term pulls generated samples toward the target sample cloud, while the repulsion term prevents generated particles from collapsing onto a few modes. In this sense, the method is related in spirit to kernel-based distribution shaping and MMD-like matching, although we use it here as a practical training rule rather than derive a formal discrepancy minimization objective.

In the simulations, the drift scale is $\eta = 1.0$, the repulsive weight is $\lambda = 1.0$, the minimum kernel bandwidth is 0.2, and the maximum drift norm is 2.0. These settings were chosen because they produced stable training behavior and materially improved the early AWGN drifting baselines.

The major operational advantage is straightforward: Once training is complete, sampling requires only a single evaluation of $g_\theta(x, z)$. In residual mode, this produces a channel residual sample $\hat{e}$ followed by reconstruction $\hat{y} = x + \hat{e}$. In direct mode, it produces $\hat{y}$ directly. There is no reverse chain, no ODE solver, and no per-sample iterative schedule at inference.

a) Conditioning-aware kernel variant.: For the enhanced direct-output drifting model, we replace the target-only kernel by a condition-aware product kernel

$$k((x_i, y_i), (x_j, y_j)) = k_x(x_i, x_j) k_y(y_i, y_j),$$

so that attraction and repulsion depend on similarity in both the channel input and the output space. In the direct-output setting, the target kernel can also incorporate residual similarity, yielding a drift field that is more closely aligned with the conditional channel structure. The generator architecture is unchanged.

III. EXPERIMENTAL SETUP

A. Channel Models

We simulate four channel families. All channels are implemented on real-valued vectors. For AWGN and Rayleigh, we use real-valued channel inputs and output vectors of length 7. For SSPA and OptFib, the real-valued signal is interpreted in paired I/Q coordinates, with dimensions 8 and 2, respectively. Noise is independent across real components unless the channel induces coupling.

AWGN:

$$y = x + n, \quad n \sim \mathcal{N}(0, \sigma^2 I). \quad (9)$$

This is the standard additive white Gaussian noise reference channel, serving as the simplest near-identity baseline.

Rayleigh: The transmitted symbol is multiplied elementwise by a random fading amplitude and then corrupted by additive Gaussian noise:

$$y = h \odot x + n, \quad (10)$$

where $\odot$ denotes elementwise multiplication,

$$h_k = \frac{1}{\sqrt{2}} \sqrt{u_k^2 + v_k^2}, \quad u_k, v_k \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1), \quad (11)$$

and $n \sim \mathcal{N}(0, \sigma^2 I)$.

Each real component is therefore scaled by an independent Rayleigh-distributed magnitude before noise is added. This channel is multiplicative rather than additive and is correspondingly less aligned with a pure residual perturbation view.

SSPA: The input is passed through a smooth, solid-state power amplifier nonlinearity before additive noise is applied. In this case, the real-valued representation is interpreted as paired I/Q components. Let

$$r = \sqrt{x_I^2 + x_Q^2} \quad (12)$$

denote the input amplitude. The nonlinear amplitude law used in the benchmark is

$$a(r) = \frac{G}{\left(1 + \left(\frac{Gr}{A_0}\right)^{2p}\right)^{1/(2p)}}, \quad (13)$$

with default parameters $p = 3$, $A_0 = 1.5$, and $G = 5.0$. The noiseless amplifier output is $\tilde{x} = a(r)x$, and the observed channel output is

$$y = \tilde{x} + n, \quad n \sim \mathcal{N}\left(0, \frac{\sigma^2}{2} I\right). \quad (14)$$

This channel introduces amplitude-dependent nonlinear distortion while preserving the input direction in the I/Q plane.

OptFib: We include a simplified memoryless optical-fiber proxy channel following [14]. As in SSPA, the real-valued state is interpreted as an I/Q pair. Over K_{step} propagation steps, the state undergoes an amplitude-dependent Kerr-style phase rotation followed by additive Gaussian perturbation at each step. Writing $x^{(k)} = [x_I^{(k)}, x_Q^{(k)}]^\top$, the nonlinear phase is

$$\phi^{(k)} = \gamma \left(\left(x_I^{(k)}\right)^2 + \left(x_Q^{(k)}\right)^2 \right) \frac{L}{K_{\text{step}}}, \quad (15)$$

and the next state is obtained by rotating $x^{(k)}$ by $\phi^{(k)}$ in the I/Q plane and adding Gaussian noise with per-step standard deviation

$$\sigma_n = \sqrt{\frac{10(P_n - 30)/10}{2K_{\text{step}}}}. \quad (16)$$

B. Training Protocol

This benchmark protocol closely follows the setup in [2].

a) Channel settings: For AWGN and Rayleigh, the setting is $n = 7$, while for SSPA it is $n = 8$. The corresponding $(E_b/N_0, R)$ pairs are (5 dB, 4/7) for AWGN, (12 dB, 4/7) for Rayleigh, and (8 dB, 6/8) for SSPA, yielding noise standard deviations 0.5260, 0.2350, and 0.3251, respectively. For OptFib, the channel parameters are $L = 5000$, $\gamma = 1.27$, $K_{\text{step}} = 50$, and $P_n = -21.3$ dBm.

b) Data and optimization settings: For AWGN and Rayleigh, we use training dataset size 10^7 , batch size 5000, and 30 epochs for drifting, diffusion, and WGAN. For SSPA, we use a training dataset size 10^7 , batch size 4096, and 160 epochs for all three method families. For these three channels, we evaluate each method on 10^6 samples and use

128 sliced Wasserstein distance (SWD) projections. OptFib is benchmarked separately with training dataset size 120,000, batch size 512, 60 training epochs, evaluation size 100,000, and 256 SWD projections.

c) *Model settings*: Drifting uses a conditional MLP with latent dimension 16, hidden width 128, two hidden layers, SiLU activations, and Adam with learning rate 10^{-3} . The drifting objective uses drift scale 1.0, median-heuristic bandwidth selection, minimum bandwidth 0.2, maximum drift norm 2.0, and repulsive weight 1.0. We train both parameterizations: direct drifting with target y and residual drifting with target $e = y - x$.

For AWGN, Rayleigh, and SSPA, diffusion models $p(y|x)$ directly with v -prediction, cosine-ZF variance schedule, 100 diffusion steps, and EMA decay 0.9. Hidden widths are 110, 128, and 110, respectively. AWGN and Rayleigh use a two-stage learning-rate schedule (10 epochs at 10^{-3} , then 20 epochs at 10^{-4}), while SSPA uses a constant 10^{-4} for 160 epochs. We report DDPM and DDIM with 100, 50, 20, and 10 denoising steps. For OptFib, diffusion uses residual modeling with ϵ -prediction, standard cosine schedule, 100 steps, hidden width 128, learning rate 10^{-3} , and EMA decay 0.995.

WGAN uses a conditional Wasserstein objective with RM-Sprop at 10^{-4} for generator and critic, five critic updates per generator update, and weight clipping at 0.01. Hidden widths are 128 (AWGN) and 256 (Rayleigh/SSPA). OptFib uses width 128.

d) *Experimental Fairness*: The benchmark is intended as a practical comparative study rather than a strict matched-capacity comparison. Architectures and training schedules were chosen as stable implementations for each model family, rather than enforcing identical parameter counts or compute budgets across drifting, diffusion, and WGAN. A more controlled matched-capacity study is left for future work.

C. Evaluation Metric

We focus on generator-level distributional accuracy and timing rather than full end-to-end communication metrics such as SER or E_b/N_0 curves, to reduce encoder-decoder dependency in the comparison. The primary metric is SWD, computed in the target space used by each benchmark path. For direct-output models, we report $\text{SWD}(\{\hat{y}_i\}, \{y_i\})$. For residual models, we report $\text{SWD}(\{\hat{e}_i\}, \{e_i\})$. Given true samples $P = \{v_i\}_{i=1}^N$ and generated samples $Q = \{\hat{v}_i\}_{i=1}^N$, SWD is estimated by drawing random unit directions θ_m , projecting both clouds onto those directions, computing the one-dimensional Wasserstein distance for each projection, and averaging over projections:

$$\text{SWD}(P, Q) = \frac{1}{M} \sum_{m=1}^M W_1(\{\langle \theta_m, v_i \rangle\}_{i=1}^N, \{\langle \theta_m, \hat{v}_i \rangle\}_{i=1}^N).$$

Thus, SWD compares the generated and true sample clouds through many one-dimensional views and captures geometric mismatch more faithfully than a purely bin-based marginal comparison. In our experiments, SWD is estimated using 128 random one-dimensional projections on AWGN, Rayleigh,

SSPA, and 256 projections on the separate OptFib benchmark. SWD is used primarily for comparability with prior diffusion-channel benchmarks [1], [2]. Compared with histogram-based L_1 distance, it also better captures sample geometry in $\mathbb{R}^n$. While SWD provides a geometry-aware comparison of conditional output distributions, it does not directly capture decision-relevant behavior such as error rates, which depend on the joint structure of (x, y) .

IV. RESULTS

A. Main SWD Benchmark Across Channels

Table I reports the SWD benchmark across AWGN, Rayleigh, SSPA, and OptFib. Direct output-space SWD is used throughout the table except for the additional residual-native row “Drifting (res., e)”, which reports residual-space SWD for the residual parameterization. DDIM-100 achieves the lowest direct-space SWD on AWGN, Rayleigh, and SSPA, while DDPM is strongest on OptFib. Among the one-shot generators under direct-space evaluation, direct-output drifting is consistently strongest across all four channels, whereas residual drifting degrades substantially once judged against the full $p(y|x)$ relation. The additional residual-space row makes clear that residual drifting remains strong in its native target space. Reduced-step DDIM variants degrade systematically as the number of sampling steps is reduced. This indicates that residual drifting is best interpreted as a specialized residual-matching surrogate rather than a uniformly strong full output-space channel model.

Timing results follow the expected pattern: Drifting and WGAN operate at roughly 0.38 to 0.42 μs per sample, whereas DDPM and DDIM range from about 0.01 to 0.13 ms, depending on the sampler and the number of denoising steps. Thus, even when diffusion achieves the strongest SWD, one-shot generators retain a substantial practical latency advantage. All inference timings were measured on the same CUDA device under identical inference conditions, with fixed batching and warm-up runs before measurement.

Figure 3 reports symbolic AWGN SER for a (4, 7) autoencoder trained through analytic and learned channel surrogates. Under this downstream metric, diffusion and WGAN remain closest to the analytic-channel baseline. The comparison between standard direct drifting and the conditioning-aware direct variant shows that kernel design materially improves downstream performance, although diffusion and WGAN remain strongest in this symbolic AWGN setting. This result also highlights that generator-level SWD, at least for drifting, does not map one-to-one to downstream communication performance.

Remark on metric limitations. While SWD provides a geometry-aware comparison of conditional output distributions, it does not directly capture decision-relevant behavior such as error rates. In particular, downstream communication metrics depend on the joint structure of (x, y) , whereas SWD is evaluated in output space.

TABLE I
TEN-SEED SWD BENCHMARK ACROSS ALL CHANNELS. LOWER IS BETTER. DIRECT OUTPUT-SPACE SWD IS REPORTED UNLESS THE ROW LABEL EXPLICITLY INDICATES RESIDUAL-SPACE SWD. THE RESIDUAL-SPACE ROW IS INCLUDED FOR DIAGNOSTIC PURPOSES ONLY.

Method	AWGN	Rayleigh	SSPA	OptFib
<i>One-shot generators</i>				
Drifting (dir.)	0.0100 $\pm$ 0.0007	0.0085 $\pm$ 0.0012	0.0060 $\pm$ 0.0008	0.0275 $\pm$ 0.0104
Drifting (res., y)	0.0670 $\pm$ 0.0138	0.3681 $\pm$ 0.0031	0.4812 $\pm$ 0.0022	0.3864 $\pm$ 0.0211
Drifting (res., e)[residual space]	0.0097 $\pm$ 0.0009	0.0062 $\pm$ 0.0009	0.0130 $\pm$ 0.0003	0.0463 $\pm$ 0.0082
Paper WGAN	0.0198 $\pm$ 0.0050	0.0171 $\pm$ 0.0050	0.0287 $\pm$ 0.0156	0.0414 $\pm$ 0.0179
<i>Diffusion samplers</i>				
DDPM	0.0070 $\pm$ 0.0004	0.0079 $\pm$ 0.0008	0.0032 $\pm$ 0.0005	0.0088 $\pm$ 0.0019
DDIM-100	0.0042 $\pm$ 0.0006	0.0044 $\pm$ 0.0007	0.0024 $\pm$ 0.0004	0.0584 $\pm$ 0.0148
DDIM-50	0.0064 $\pm$ 0.0004	0.0066 $\pm$ 0.0010	0.0030 $\pm$ 0.0005	—
DDIM-20	0.0143 $\pm$ 0.0003	0.0146 $\pm$ 0.0014	0.0056 $\pm$ 0.0006	—
DDIM-10	0.0264 $\pm$ 0.0004	0.0271 $\pm$ 0.0013	0.0092 $\pm$ 0.0008	—

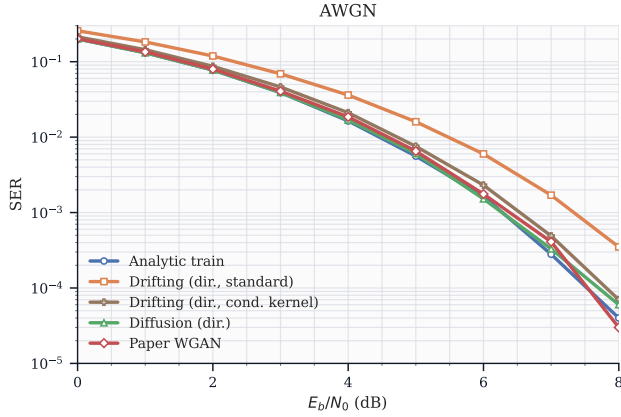


Fig. 3. End-to-end symbolic AWGN SER for a (4, 7) autoencoder trained through analytic and learned channel surrogates. Diffusion and WGAN remain closest to the analytic baseline, while the conditioning-aware direct drifting variant improves upon standard direct drifting. This illustrates that generator-level metrics such as SWD do not fully capture decision-relevant structure.

V. CONCLUSION

This paper presented a comprehensive experimental study of conditional drifting models for learned channel simulation and compared them against conditional diffusion and WGAN baselines on AWGN, Rayleigh, SSPA, and optical-fiber proxy channels. The results show a clear tradeoff between fidelity and latency. Diffusion provides the strongest distributional accuracy overall, while drifting remains competitive and retains the one-shot advantage with substantially lower inference cost. Under direct output-space evaluation, direct-output drifting is the strongest one-shot generator across the main benchmark. The additional residual-space row shows that residual drifting remains strong when evaluated in its native residual target space, but that this advantage does not transfer to full output-space matching. These results suggest that drifting is a practically relevant direction for learned channel simulation, especially when low-latency generation matters. More broadly, these experiments highlight the importance of understanding how generator-level SWD relates to downstream communication performance and motivate further study of channel-specific kernel design for improving direct-output drifting.

Future work should extend the comparison to broader channel families, including MIMO, memory, and more realistic optical channels, together with downstream communication metrics. A more formal analysis of the drift objective and its kernel dependence is left for future work.

REFERENCES

- [1] M. Kim, R. Fritschek, and R. F. Schaefer, “Diffusion models for accurate channel distribution generation,” *arXiv preprint arXiv:2309.10505*, 2023.
- [2] —, “Robust generation of channel distributions with diffusion models,” in *IEEE International Conference on Communications (ICC)*, 2024.
- [3] T. Lee, J. Park, H. Kim, and J. G. Andrews, “Generating high dimensional user-specific wireless channels using diffusion models,” *arXiv preprint arXiv:2409.03924*, 2024.
- [4] X. Gong, X. Liu, A. A. Lu, X. Gao, X. G. Xia, C.-X. Wang, and X. You, “Digital twin of channel: Diffusion model for sensing-assisted statistical channel state information generation,” *IEEE Transactions on Wireless Communications*, vol. 24, no. 5, pp. 3805–3821, 2025.
- [5] X. Zhou, L. Liang, J. Zhang, P. Jiang, Y. Li, and S. Jin, “Generative diffusion models for high dimensional channel estimation,” *IEEE Transactions on Wireless Communications*, vol. 24, no. 7, pp. 5840–5854, 2025.
- [6] B. Fesl, M. Baur, F. Strasser, M. Joham, and W. Utschick, “Diffusion-based generative prior for low-complexity MIMO channel estimation,” *IEEE Wireless Communications Letters*, vol. 13, no. 12, pp. 3493–3497, 2024.
- [7] Z. Chen, H. Shin, and A. Nallanathan, “Generative diffusion model-based variational inference for MIMO channel estimation,” *IEEE Transactions on Communications*, vol. 73, no. 10, pp. 9254–9269, 2025.
- [8] N. Zilberstein, A. Swami, and S. Segarra, “Joint channel estimation and data detection in massive MIMO systems based on diffusion models,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024.
- [9] H. Fu, W. Si, and R. Liu, “Conditional denoising diffusion-based channel estimation for fast time-varying MIMO-OFDM systems,” *Digital Signal Processing*, vol. 164, p. 105283, 2025.
- [10] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” in *International Conference on Learning Representations*, 2021.
- [11] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever, “Consistency models,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research. PMLR, 2023. [Online]. Available: <https://proceedings.mlr.press/v202/song23a.html>
- [12] T. Yin, M. Gharbi, R. Zhang, E. Shechtman, F. Durand, W. T. Freeman, and T. Park, “One-step diffusion with distribution matching distillation,” in *CVPR*, 2024.
- [13] M. Deng, H. Li, T. Li, Y. Du, and K. He, “Generative modeling via drifting,” *arXiv preprint arXiv:2602.04770*, 2026.
- [14] S. Li, C. Häger, N. Garcia, and H. Wymeersch, “Achievable information rates for nonlinear fiber communication via End-to-End autoencoder learning,” in *Proc. European Conference on Optical Communication (ECOC)*, Rome, Italy, Sep. 2018, pp. 1–3.